

LLM COMPILER: Foundation Language Models for Compiler Optimization

Chris Cummins*

Meta
Menlo Park, USA

Volker Seeker*

Meta
Menlo Park, USA

Dejan Grubisic

Meta
Menlo Park, USA

Baptiste Roziere

Meta
Paris, France

Jonas Gehring

Meta
Paris, France

Gabriel Synnaeve

Meta
Paris, France

Hugh Leather*

Meta
Menlo Park, USA

Abstract

Large Language Models (LLMs) have demonstrated remarkable capabilities across a variety of software engineering and coding tasks. However, their application in the domain of code and compiler optimization remains underexplored. Training LLMs is resource-intensive, requiring substantial GPU hours and extensive data collection, which can be prohibitive. To address this gap, we introduce LLM COMPILER, a suite of robust, openly available, pre-trained models specifically designed for compiler tasks. Built on the foundation of CODE LLAMA, LLM COMPILER enhances the understanding of compiler intermediate representations (IRs), assembly language, and optimization techniques. The models have been trained on a vast corpus of 546 billion tokens of LLVM-IR and assembly code and have undergone instruction fine-tuning to interpret compiler behavior.

To demonstrate the utility of these research tools, we also present fine-tuned versions of the models with enhanced capabilities in optimizing code size and disassembling from x86_64 and ARM assembly back into LLVM-IR. These achieve 77 % of the optimising potential of an autotuning search, and 45 % disassembly round trip (14 % exact match).

LLM COMPILER is released under a bespoke commercial license to allow wide reuse and is available in two sizes: 7 billion and 13 billion parameters. Our aim is to provide scalable, cost-effective foundational models for further research and development in compiler optimization by both academic researchers and industry practitioners. Since we released LLM COMPILER the community has quantized, repackaged, and downloaded the models over 250k times.

*Core contributors. Contact cummins@meta.com or vseeker@meta.com.



This work is licensed under a Creative Commons Attribution 4.0 International License.

CC '25, Las Vegas, NV, USA

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1407-8/25/03

<https://doi.org/10.1145/3708493.3712691>

CCS Concepts: • Software and its engineering → Compilers; • Computing methodologies → Supervised learning.

Keywords: Large Language Models, Compiler Optimization, Code Optimization, Pre-trained Models, LLVM-IR

ACM Reference Format:

Chris Cummins, Volker Seeker, Dejan Grubisic, Baptiste Roziere, Jonas Gehring, Gabriel Synnaeve, and Hugh Leather. 2025. LLM COMPILER: Foundation Language Models for Compiler Optimization. In *Proceedings of the 34th ACM SIGPLAN International Conference on Compiler Construction (CC '25)*, March 1–2, 2025, Las Vegas, NV, USA. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3708493.3712691>

1 Introduction

There is increasing interest in large language models (LLMs) for software engineering tasks including code generation, code translation, and code testing. Models such as StarCoder [37], CODE LLAMA [46], and GPT-4 [40] have a good statistical understanding of code and can suggest likely completions for unfinished code, making them useful for editing and creating software. However, there is little emphasis on training specifically to optimize code. Publicly available LLMs can be prompted to make minor tweaks to a program such as tagging variables to be stored as registers, and will even attempt more substantial optimizations like vectorization, though they easily become confused and make mistakes, frequently resulting in incorrect code.

Prior works on machine learning-guided code optimization have used a range of representations from hand-built features [57] to graph neural networks (GNNs) [35]. However, in all cases, the way the input program is represented to the machine learning algorithm is incomplete, losing some information along the way. For example, *MLGO* [55] uses numeric features to provide hints for function inlining, but cannot faithfully reproduce the call graph or control flow. *ProGraML* [7] forms graphs of the program to pass to a GNN, but excludes the values of constants and some type information which prevents reproducing instructions with fidelity.

In contrast, LLMs can accept source programs, as is, with a complete, lossless representation. Using text as the input and

output representation for a machine learning optimizer has desirable properties: text is a universal, portable, and accessible interface, and unlike prior approaches is not specialized to any particular task.

However, training LLMs incurs high cost in both compute and data. For example, training CODE LLAMA’s models consumed 1.4 M A100 GPU hours to train, and curating the vast amounts of training data (hundreds of billions of tokens) can be challenging. These costs are often prohibitive to researchers in the field and this blocks advances that might otherwise be possible.

To address this issue, we present LLM COMPILER¹, a family of foundation models that have already been trained to understand the semantics of compiler IRs and assemblies and to emulate the compiler, allowing for easy fine-tuning with minimal data² for specific downstream compiler optimization tasks. Building upon CODE LLAMA, we extend its capabilities to encompass compiler optimization and reasoning.

The training pipeline for LLM COMPILER is illustrated in Figure 1. We extend CODE LLAMA with additional pretraining on a vast corpus of assembly codes and compiler IRs, and then instruction fine-tune on a bespoke *compiler emulation* dataset to better reason about code optimization. Our intention with releasing these models is to provide a foundation for researchers and industry practitioners to further develop code optimization models.

To demonstrate their utility as compiler research tools, we adapt the models for two downstream tasks: tuning compiler flags to optimize for code size, and disassembling x86_64 and ARM assembly to LLVM-IR. We also release these LLM COMPILER FTD models to the community under the same bespoke commercial license. Compared to the autotuning technique on which it was trained, LLM COMPILER FTD achieves 77 % of the optimizing potential without the need for any additional compilations. When disassembling, LLM COMPILER FTD creates correct disassembly 14 % of the time. On both tasks LLM COMPILER FTD models significantly outperform comparable LLMs CODE LLAMA and GPT-4 Turbo.

Our work aims to establish a scalable, cost-effective foundation for further research and development in compiler optimization, catering to both academic researchers and industry practitioners. By providing access to pre-trained models in two sizes (7 billion and 13 billion parameters) and demonstrating their effectiveness through fine-tuned versions, LLM Compiler paves the way for exploring the untapped potential of LLMs in the realm of code and compiler optimization.

2 Overview

Figure 1 shows an overview of our approach. LLM COMPILER models target compiler optimization. They are available in

¹<https://huggingface.co/collections/facebook/llm-compiler-667c5b05557fe99a9edd25cb>

²This can be done on small clusters or even single GPUs as shown in previous work [12, 28].

Table 1. Training datasets used.

Dataset	Sampling prop.	Epochs	Disk size
IR and assembly pretraining (401 billion tokens)			
Code	85.00 %	1.000	872 GB
Natural language related to code	14.00 %	0.019	942 GB
Natural language	1.00 %	0.001	938 GB
Compiler emulation (additional 145 billion tokens)			
Compiler emulation	85.00 %	1.702	175 GB
Code	13.00 %	0.055	872 GB
Natural language related to code	1.80 %	0.001	942 GB
Natural language	0.20 %	6.9×10^{-5}	938 GB
Flag tuning fine-tuning (additional 84 billion tokens)			
Flag tuning	85.00 %	1.700	103 GB
Compiler emulation	11.73 %	0.136	175 GB
Code	2.84 %	0.007	872 GB
Natural language related to code	0.40 %	1.1×10^{-4}	942 GB
Natural language	0.03 %	8.8×10^{-6}	938 GB
Disassembly fine-tuning (additional 80 billion tokens)			
Disassembly	85.00 %	1.707	88 GB
Flag tuning	4.68 %	0.089	103 GB
Compiler emulation	8.07 %	0.089	175 GB
Code	1.96 %	0.004	872 GB
Natural language related to code	0.27 %	7.5×10^{-5}	942 GB
Natural language	0.03 %	5.7×10^{-6}	938 GB

two model sizes: 7B and 13B parameters. The LLM COMPILER models are initialized with CODE LLAMA model weights of the corresponding size and trained on an additional 546 B tokens of data comprising mostly compiler intermediate representations and assembly code. We then further train LLM COMPILER FTD models using an additional 164 B tokens of data for two downstream compilation tasks: flag tuning and disassembly. At all stages of training a small amount of code and natural language data from previous stages is used to help retain the capabilities of the base model.

3 Specializing LLMs for Compilers

3.1 Pretraining on Assembly Code and Compiler IRs

The data used to train coding LLMs is typically weighted heavily towards high level source and scripting languages. For example, Python, PHP, and JavaScript make up 27.4 % of The Stack [31], while Assembly and compiler IRs make up only 0.08 %. To build an LLM with a good understanding of these languages we initialize LLM COMPILER models with the weights of CODE LLAMA and then train for 401 billion tokens on a compiler-centric dataset composed mostly of assembly code and compiler IRs, shown in Table 1.

LLM COMPILER is trained predominantly on compiler intermediate representations and assembly code generated by LLVM [32] version 17.0.6. These are derived from the same dataset of publicly available code used to train CODE LLAMA. We summarize this dataset in Table 2. As in CODE LLAMA, we also source a small proportion of training batches from natural language datasets.

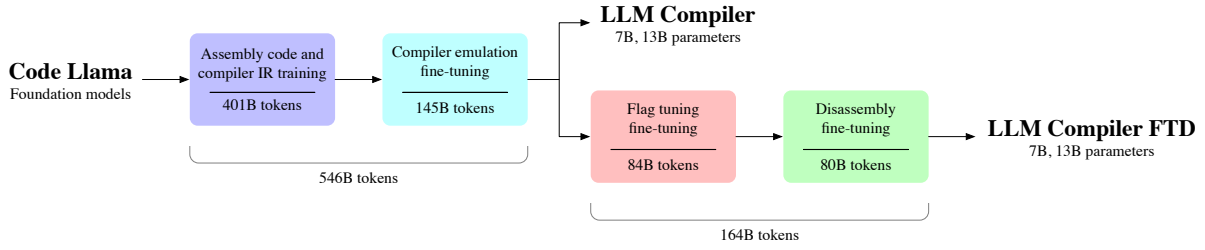


Figure 1. LLM COMPILER models are specialized from CODE LLAMA by training on 546 billion tokens of compiler-centric data in two stages. In the first stage the models are trained predominantly on unlabelled compiler IRs and assembly code. In the next stage the models are instruction fine-tuned to predict the output and effect of optimizations. LLM COMPILER FTD models are then further fine-tuned on 164 billion tokens of downstream flag tuning and disassembly task datasets, for a total of 710 billion training tokens. During each of the four stages of training, 15 % of data from the previous tasks is retained.

Table 2. Composition of data used for initial IR and assembly pretraining. LLM COMPILER is trained on a near-even split of IR and assembly code, predominantly targeting x86-64 architecture, with some 64-bit ARM, and a small amount of CUDA.

	Items	Tokens	Disk size
LLVM-IR	10.7 M	185 B	432 GB
Assembly	10.1 M	216 B	440 GB
Total	20.8 M	401 B	872 GB

(a) Language

	Items	Tokens	Disk size
x86_64-unknown-linux-gnu	17.3 M	340.3 B	738 GB
aarch64-unknown-linux-gnu	3.5 M	60.5 B	133 GB
nvptx64-nvidia-cuda	9.2 k	146 M	286 MB
Total	20.8 M	401 B	872 GB

(b) Target

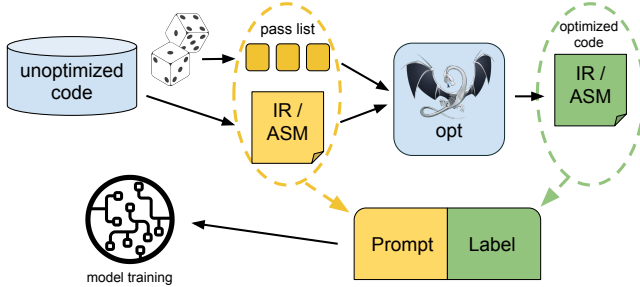


Figure 2. To improve understanding of how compiler optimizations work, we train models on a *compiler emulation* task. Unoptimized code samples and random pass lists are given to `opt` to generate optimized code (IR or assembly). Pass list and input code are taken together as prompt while the generated output code is used as label.

3.2 Instruction Fine-Tuning for Compiler Emulation

To understand the mechanism of code optimization we instruction fine-tune LLM COMPILER models to emulate compiler optimizations, illustrated in Figure 2. The idea is to

```
opt -o a.bc -p 'module(default<0z>),module(iroutliner)'
clang a.bc -o a.o
size a.o
```

Figure 3. Commands used to apply an optimization pipeline comprising `-Oz` passes followed by IR outlining to an unoptimized IR. Binary size is the sum of `.TEXT` and `.DATA` section sizes of the object file as reported by `size`.

generate from a finite set of unoptimized seed programs a large number of examples by applying randomly generated sequences of compiler optimizations to these programs. We then train the model to predict the code generated by the optimizations. We also train the model to predict the code size after the optimizations have been applied.

Task Specification. Given unoptimized LLVM-IR (as emitted by the `clang` frontend), a list of optimization passes, and a starting code size, generate the resulting code and the resulting code size after those optimizations have been applied.

There are two flavors of this task: in the first the model is expected to output compiler IR, in the second the model is expected to output assembly code. The input IR, optimization passes, and code size are the same for both flavors. The prompt dictates the required output format.

Code Size. We use two metrics for code size: the number of IR instructions, and *binary size*, computed by summing the size of the `.TEXT` and `.DATA` sections of the IR or assembly after lowering to an object file. We exclude `.BSS` section from our binary size metric since it does not affect on-disk size.

Optimization Passes. In this work we target LLVM 17.0.6 and use the New Pass Manager [44] which classifies passes for different levels such as *module*, *function*, *loop*, etc. as well as transformation and analysis passes. Transformation passes change given input IR while analysis passes generate information that influence subsequent transformations.

Of the 346 possible pass arguments for `opt`, we select 167 to use. This includes each of the default optimization pipelines

(e.g. `module(default<Oz>)`), individual optimization transform passes (e.g. `module(constmerge)`), but excludes non-optimization utility passes (e.g. `module(dot-callgraph)`) and transformations passes that are not semantics preserving (e.g. `module(internalize)`). We exclude analysis passes since they have no side effects and we rely on the pass manager to inject dependent analysis passes as needed. For passes that accept parameter arguments we use the default values (e.g. `module(licm<allowspeculation>)`). We used LLVM’s *opt* tool to apply pass lists and *clang* to lower the resulting IR to object file. Listing 3 shows the commands used.

Dataset. We generated the compiler emulation dataset by applying random lists of between 1 and 50 optimization passes to unoptimized programs summarized in Table 2. The length of each pass list was selected uniformly at random. Pass lists were generated by uniformly sampling from the set of 167 passes described above. Pass lists which resulted in compiler crashes or timed out after 120 seconds were excluded.

4 Demonstration on Downstream Tasks

To showcase the value of LLM COMPILER as a tool for compiler and programming research, we fine-tune it on two downstream tasks: flag tuning and decompilation. In Section 6, we show the improvements this affords over training from scratch. We also release these fine-tuned models which are useful tools in their own rights.

4.1 Compiler Optimization Flag Tuning

Manipulating compiler flags is well known to have a considerable impact on both runtime performance and code size [16]. We fine-tune LLM COMPILER models on the downstream task of selecting flags for LLVM’s IR optimization tool *opt* to produce the smallest code size. Machine learning approaches to flag tuning have shown good results previously, but struggle with generalizing across different programs [10]. Previous works usually need to compile new programs tens or hundreds of times to try out different configurations and find out the best-performing option. We train and evaluate LLM COMPILER FTD models on the zero-shot version of this task by predicting flags to minimize code size of unseen programs. Our approach is agnostic to the chosen compiler and optimization metric, and we intend to target runtime performance in the future. For now, optimizing for code size simplifies the collection of training data.

Task Specification. We present the models with an unoptimized LLVM-IR (as emitted by the *clang* frontend) and ask it to produce a list of *opt* flags that should be applied to it, the binary size before and after these optimizations are applied, and the output code. If no improvement can be made over the input code, a short output message is generated that contains only the unoptimized binary size. We

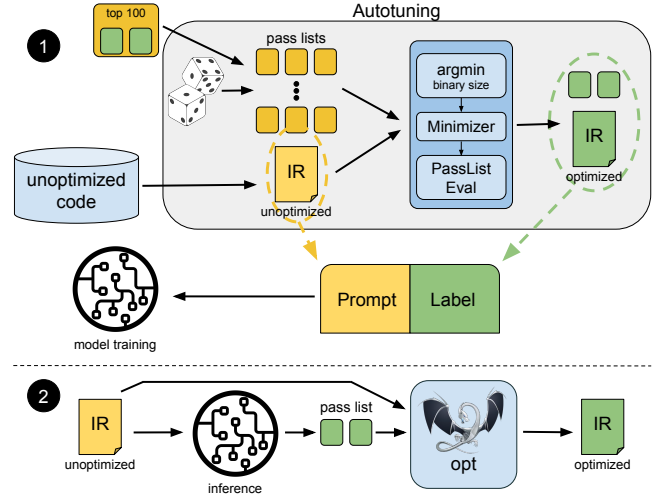


Figure 4. Overview of our approach, showing the model input (Prompt) and output (Label) during training ① and inference ②. The prompt contains unoptimized code. The label contains an optimization pass list, binary size, and the optimized code. To generate the label for the training prompt, the unoptimized code is compiled against multiple random pass lists. The pass list achieving the minimum binary size is selected, minimized and checked for correctness with PassListEval. The final pass list together with its corresponding optimized IR are used as label during training. In a last step, the top 100 most often selected pass lists are broadcast among all programs. For deployment we generate only the optimization pass list which we feed into the compiler, ensuring that the optimized code is correct.

used the same constrained set of optimization passes as in the compiler emulation task, and compute binary size in the same manner.

Figure 4 illustrates the process used to generate training data (described below) and how the model is used for inference. Only the generated pass list is needed at evaluation time. We extract the pass list from the model output and run *opt* using the given arguments. We can then evaluate the accuracy of the model predicted binary sizes and optimized output code, but those are auxiliary learning tasks not required for use.

Correctness. LLVM’s optimizer is not free from bugs and running optimization passes in unexpected or untested orders may expose subtle correctness errors that undermine the utility of the model. To mitigate this risk we developed *PassListEval*, a tool to help in automatically identifying pass lists that break program semantics or cause compiler crashes. An overview of the tool is shown in Figure 5. *PassListEval* accepts as input a candidate pass list and evaluates it over a suite of 164 self-testing C++ programs, taken from HumanEval-X [61]. Each program contains a reference solution for a programming challenge, e.g. “Check if in given

vector of numbers, are any two numbers closer to each other than given threshold”, and a suite of unit tests that validate correctness. We apply the candidate pass lists to the reference solution, and then link them against the test suites to produce a binary. When executed, the binary will crash if any of the tests fail. If any binary crashes, or if any of the compiler invocations fail, we reject the candidate pass list.

Dataset. We trained LLM COMPILER FTD models on a dataset of flag tuning examples derived from 4.5M of the unoptimized IRs used for pretraining. To generate the example optimal pass list for each program we ran an extensive iterative compilation process depicted in Figure 4 and outlined below:

- 1) We used large-scale *random search* to generate an initial candidate best pass list for the programs. For each program we independently generated random lists of up to 50 passes by uniformly sampling from the set of 167 searchable passes described previously. Every time we evaluated a pass list on a program we recorded the resulting binary size. We then pick the per-program pass lists that produced the lowest binary size. We ran 22 billion unique compilations for an average 4,877 per program.

- 2) The pass lists generated by random search may contain redundant passes that have no effect on the final outcome. Further, some pass orderings are commutative such that re-ordering then does not affect the final outcome. Since these would introduce noise in our training data, we developed a *minimization* process which we applied to each pass list. Minimization comprises three steps: redundant pass elimination, bubble sort, and insertion search. In redundant pass elimination we minimize the best pass list by iteratively removing individual passes to see if they contribute to the binary size. If not, they are discarded. This is repeated until no further passes can be discarded. Bubble sort then attempts to provide a uniform ordering for pass subsequences by sorting passes based on a key. Finally, insertion sort performs a local search by iterating over each pass in the pass list and attempting to insert each of the 167 search passes before it. If doing so improves the binary size, this new pass list is kept. The entire minimization pipeline loops until a fixed point is reached. The average pass list length is 3.84.

- 3) We apply *PassListEval*, described previously, to the candidate best pass lists. Through this we identified 167,971 of 1,704,443 unique pass lists (9.85 %) as causing compile time or runtime errors.

- 4) We broadcast the *top 100* most frequently optimal pass lists across all programs, updating the per-program best pass lists if improvements are found. After this the total number of unique best pass lists decreases from 1,536,472 to 581,076.

The autotuning pipeline outlined above produced a geometric mean 7.1 % reduction in binary size over -Oz. For our purposes, this autotuning serves as a gold standard for the optimization of each program. While the binary size

savings discovered are significant, this required 28 billion additional compilations at a computational cost of over 504,000 CPU hours. The goal of instruction fine-tuning LLM COMPILER FTD to perform the flag tuning task is to achieve some fraction of the performance of the autotuner without requiring running the compiler thousands of times.

4.2 Instruction Fine-Tuning for Disassembly

The ability to lift code from assembly back into higher level structures enables running additional optimizations on library code directly integrated with application code or porting of legacy code to new architectures. The field of decompilation has seen advancements in applying machine learning techniques to generate readable and accurate code from binary executables. Several studies explore the use of machine learning for decompilation tasks, such as lifting binaries into intermediate representations for evaluation against synthetic C programs [5], utilizing evolutionary approaches like genetic algorithms for program analysis [47], and proposing methods like XLIR for matching binary code across different programming languages [22]. Armengol-Estapé et al. [3] have trained a language model to decompile x86 assembly into high level C code. In this study, we demonstrate how LLM COMPILER FTD can learn the relationship between assembly code and compiler IR by fine-tuning it for disassembly. The task is to learn the inverse translation of clang -xir - -o - -S, shown in Figure 6.

Round Tripping. Using an LLM for disassembly causes problems of correctness. The lifted code must be verified by an equivalence checker which is not always feasible or manually verified for correctness or subjected to sufficient test cases to give confidence. However, a lower bound on correctness can be found by round-tripping. That is to say by compiling the lifted IR back into assembly, if the assembly is identical then the IR is correct. This gives an easy route to using the results of the LLM and an easy way to measure the utility of a disassembly model.

Task Specification. We provide the model with assembly code and train it to emit the corresponding disassembled IR. The context length for this task is set to 8 k tokens for the input assembly code and 8 k tokens for the output IR.

Dataset. We derive the assembly codes and IR pairs from the same dataset used in previous tasks. Our fine-tuning dataset comprises 4.7 M samples. The input IR has been optimized with -Oz before being lowered to x86 assembly.

5 Training Parameters

Data is tokenized via byte pair encoding [17], employing the same tokenizer as CODE LLAMA, Llama [53], and Llama 2 [54].

We use the same training parameters for all four stages of training. Most of the training parameters we used are the same as for the CODE LLAMA base model. We use the

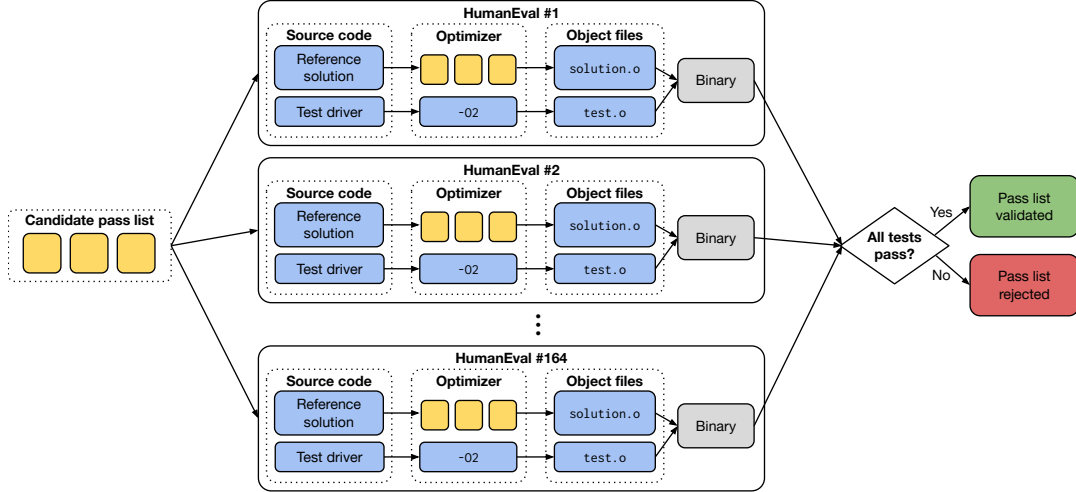


Figure 5. Validating a candidate list of optimization passes using PassListEval. The candidate pass list is applied to the reference solutions for all 164 programs in HumanEval-X. The unit tests for these reference solutions are optimized using a conservative -O2 pass pipeline to ensure correctness, and then linked against the reference solutions. The resulting binaries are executed and if any of the binaries crash during execution, or if any of the compiler invocations fail, the pass list is rejected.

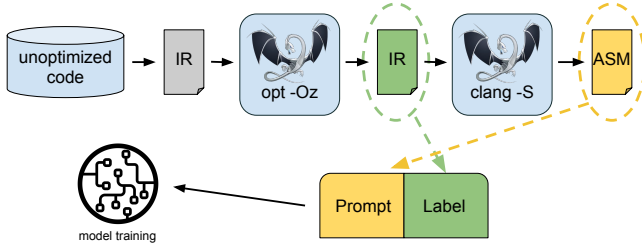


Figure 6. We train the model to understand IR-assembly relationships by teaching it to disassemble code to its corresponding IR. The IR used to label this training task was generated by optimizing an IR with the -Oz flag.

AdamW [36] optimizer with β_1 and β_2 values of 0.9 and 0.95. We use a cosine schedule with 1000 warm-up steps, and set the final learning rate to be 1/30th of the peak learning rate. Compared to the CODE LLAMA base model, we increased the context length of individual sequences from 4,096 to 16,384, but kept the batch size constant at 4 M tokens. To account for the longer context, we set our learning rate to $2e^{-5}$ and modified the parameters of the RoPE positional embeddings [50] where we reset frequencies with a base value of $\theta = 10^6$. These settings are in accordance with the long context training done for the CODE LLAMA base model. In aggregate, training all four LLM COMPILER models required 264 K GPU hours of computation on hardware of type A100-80 GB (TDP of 350-400 W).

6 Evaluation

6.1 Flag Tuning Task

Methodology. We evaluate LLM COMPILER FTD on the task of optimization flag tuning for unseen programs and compare to GPT-4 Turbo and CODE LLAMA - INSTRUCT. We

run inference on each model and extract from the model output the optimization pass list. We then use this pass list to optimize the particular program and record the binary size. The baseline is the binary size of the program when optimized using -Oz.

For GPT-4 Turbo and CODE LLAMA - INSTRUCT we append a suffix to the prompt with additional context to further describe the problem and expected output format.

All model-generated pass lists are validated using *PassListEval*, and -Oz is used as substitute if validation fails. To further validate correctness of model-generated pass lists we link the final program binaries and differential test their outputs against the outputs of the benchmark when optimized using a conservative -O2 optimization pipeline.

Dataset. We evaluate on 2,398 test prompts extracted from the MiBench benchmark suite [25]. To generate these prompts we take all of the 713 translation units that make up the 24 MiBench benchmarks and generate unoptimized IRs from each. We then format them as prompts. If the resulting prompt exceeds 15 k tokens we split the LLVM module representing that translation unit into smaller modules, one for each function, using *llvm-extract*. This results in 1,959 prompts which fit within the 15 k token context window, leaving 439 translation units which do not fit. We use -Oz when for the 439 excluded translation units when computing performance scores.

Results. Table 3 shows zero-shot performance of all models on the flag tuning task. Only LLM COMPILER FTD models provide an improvement over -Oz, with the 13B parameter model marginally outperforming the smaller model, generating smaller object files than -Oz in 61 % of cases.

In some cases the model-generated pass list causes a larger object file size than -Oz. For example, LLM COMPILER FTD 13B regresses in 12 % of cases. These regressions can be avoided by simply compiling the program twice: once using the model-generated pass list, once using -Oz, and selecting the pass list which produces the best result. By eliminating regressions wrt -Oz, these -Oz *backup* scores raise the overall improvement over -Oz to 5.26 % for LLM COMPILER FTD 13B, and enable modest improvements over -Oz for CODE LLAMA - INSTRUCT and GPT-4 Turbo. Figure 7 shows the performance of each model broken down by individual benchmark.

Binary Size Accuracy. While the model-generated binary size predictions have no effect on actual compilation, we can evaluate the performance of the models at predicting binary sizes before and after optimization to give an indication of each model’s understanding of optimization. Figure 8 shows the results for unoptimized code size (optimized code size results are similar). LLM COMPILER FTD binary size predictions correlate well with ground truth, with the 7B parameter model achieving MAPE values of 0.083 and 0.225 for unoptimized and optimized binary sizes respectively. The 13B parameter model has similar MAPE values of 0.082 and 0.225. CODE LLAMA - INSTRUCT and GPT-4 Turbo binary size predictions show little correlation with ground truth. We note that the LLM COMPILER FTD errors are slightly higher for optimized code than unoptimized code. In particular, there is an occasional tendency for LLM COMPILER FTD to overestimate the effectiveness of optimization, resulting in a lower predicted binary size than actual.

Ablation Studies. Table 4 ablates the performance of models on a small holdout validation set of 500 prompts taken from the same distribution as our training data (though not used during training). We trained for flag tuning at each stage of the training pipeline from Figure 1 to compare performance. As shown, disassembly training causes a slight regression in performance from average 5.15 % to 5.12 % improvement over -Oz. We also show performance of the autotuner used for generating the training data described in Section 3. LLM COMPILER FTD achieves 77 % of the performance of the autotuner.

6.2 Disassembly Task

Methodology. We evaluate the functional correctness of LLM-generated code when disassembling assembly code to LLVM-IR. As in Section 6.1 we evaluate LLM COMPILER FTD and compare to CODE LLAMA - INSTRUCT and GPT-4 Turbo, and find that an additional prompt suffix is required to extract the best performance from these models. The suffix provides additional context about the task and the expected output format. To evaluate the performance of models we *round-trip* the model-generated disassembled IR back down to assembly. This enables us to evaluate accuracy of the disassembly by comparing the BLEU score [41] of the original

assembly against the round-trip result. A lossless and perfect disassembly from assembly to IR will have a round-trip BLEU score of 1.0 (*exact match*).

Dataset. We evaluate on 2,015 test prompts extracted from the MiBench benchmark suite. We took the 2,398 translation units used for the flag tuning evaluation above and generated disassembly prompts. We then filtered the prompts on a maximum 8 k token length, allowing 8 k tokens for the model output, leaving 2,015.

Results. Table 5 shows performance of the models on the disassembly task. LLM COMPILER FTD 7B has a slightly higher round-trip success rate than LLM COMPILER FTD 13B, but LLM COMPILER FTD 13B has the highest accuracy of round-tripped assembly (*round trip BLEU*) and most frequently produces a perfect disassembly (*round trip exact match*). CODE LLAMA - INSTRUCT and GPT-4 Turbo struggle with generating syntactically correct LLVM-IR. Figure 9 shows the distribution of round-trip BLEU scores for all models.

Ablation Studies. Table 6 ablates the performance of models on a small holdout validation set of 500 prompts taken from the MiBench dataset used previously. We trained for disassembly at each stage of the training pipeline from Figure 1 to compare performance. Round trip rate is highest when going through the whole stack of training data and drops consistently with every training stage, though round trip BLEU varies little with each stage.

6.3 Foundation Model Tasks

Methodology. We ablate LLM COMPILER models on the two foundation model tasks of next-token prediction and compiler emulation. We perform this evaluation at each stage of the training pipeline to see how training for each successive task affects performance. For next-token prediction we compute perplexity on a small sample of LLVM-IR and assembly code from all optimization levels. We evaluate compiler emulation using two metrics: whether the generated IR or assembly code compiles, and whether the generated IR or assembly code is an exact match for what the compiler would produce.

Dataset. For next-token prediction we use a small holdout set of validation data that is drawn from the same distribution as our training data but has not been used for training. We use a mixture of optimization levels including unoptimized code, code optimized with -Oz, and randomly generated pass lists. For compiler emulation we evaluate using 500 prompts generated from MiBench using randomly pass lists generated in the manner described in Section 3.2.

Results. Table 7 shows LLM COMPILER FTD’s performance across all training stages on the two foundation model training tasks of next-token prediction and compiler emulation.

Table 3. Comparison of model performance when flag tuning 2,398 object files from MiBench. *Overall improvement* scores include 443 object files which do not fit in the context window of LLM COMPILER FTD. For GPT-4 and the CODE LLAMA models we appended a suffix to the prompt to provide additional context.

	Size	Improved	Regressed	Overall improvement over -Oz zero-shot	-Oz backup
LLM COMPILER FTD	7B	1,465	302	4.77 %	5.24 %
	13B	1,466	299	4.88 %	5.26 %
CODE LLAMA - INSTRUCT	7B	379	892	-0.49 %	0.23 %
	13B	319	764	-0.42 %	0.18 %
	34B	230	493	-0.27 %	0.15 %
GPT-4 Turbo (2024-04-09)	-	13	24	-0.01 %	0.03 %

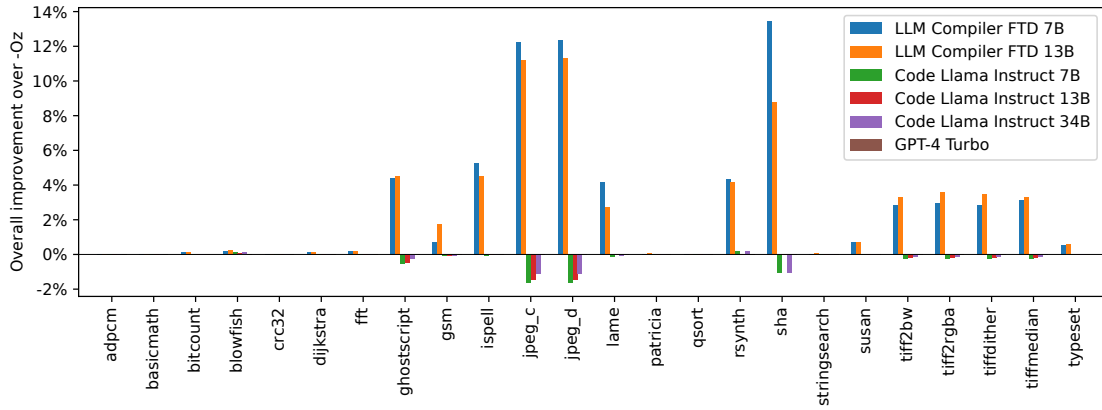


Figure 7. Object file size improvement over -Oz for each of the benchmarks in MiBench.

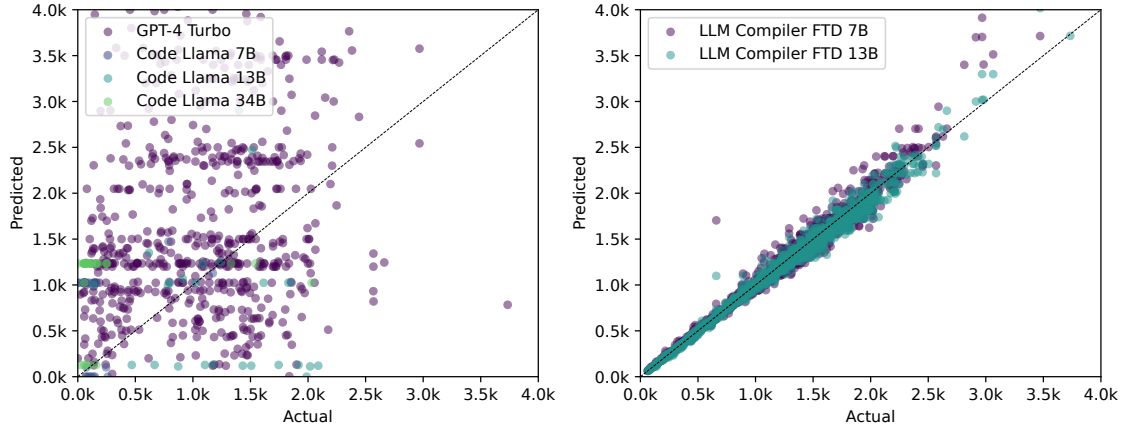


Figure 8. Accuracy of models at predicting unoptimized code size. LLM COMPILER FTD is most accurate at predicting code size. CODE LLAMA and GPT-4 Turbo, shown left, display little correlation between predicted and actual values.

Next-token prediction performance jumps sharply after CODE LLAMA, which has seen very little IR and assembly, and declines slightly with each subsequent stage of fine-tuning.

For compiler emulation, the CODE LLAMA base model and the pre-trained models perform poorly since they have not been trained on this task. The highest performance is achieved directly after compiler emulation training where 95.6% of IR and assembly generated by LLM COMPILER FTD

13B compiles, and 20% of it matches the compiler exactly. Performance declines after fine-tuning for flag tuning and disassembly.

7 Related Work

There is increasing interest in LLMs for source code reasoning and generation [27, 30]. The main enablers of progress

Table 4. Ablating the LLM COMPILER FTD training regime on the flag tuning task. All results are for 7B parameter models, evaluated on the same holdout validation set of 500 programs. The first row is LLM COMPILER FTD, the other rows are ablations.

CODE LLAMA	IR & asm pretraining	Compiler emulation	Flag tuning	Disassembly	Mean improvement	
					over -Oz	wrt. Autotuner
✓	✓	✓	✓	✓	5.12 %	77 %
✓	✓	✓	✓		5.15 %	78 %
✓	✓		✓		5.07 %	76 %
✓			✓		4.94 %	75 %
			✓		4.79 %	72 %
Autotuner					6.63 %	100 %

Table 5. Model performance at disassembling 2,015 assembly codes taken from MiBench. We use *Round trips* to evaluate the capabilities of models, by taking the IR generated by the models and attempting to lower it back to assembly. *Round trips* shows the number of disassembled IRs that can be lowered back, *Round trip BLEU* compares the round-tripped assemblies against the originals, and *Round trip exact match* is the proportion of round-tripped assemblies that are exact character-for-character matches with the input, indicating lossless round-trip from assembly up to IR and back down again.

	Size	Round trips	Round trip BLEU	Round trip exact match
LLM COMPILER FTD	7B	936	0.951	12.7 %
	13B	905	0.960	13.8 %
CODE LLAMA - INSTRUCT	7B	30	0.477	0.0 %
	13B	53	0.615	0.0 %
	34B	12	0.458	0.0 %
GPT-4 Turbo (2024-04-09)	-	127	0.429	0.0 %

Table 6. Ablating the LLM COMPILER FTD training regime on code disassembly. All results are for 7B model sizes, evaluated on holdout validation set of 500 programs. Parentheses show relative performance to the first row (i.e. LLM COMPILER FTD).

CODE LLAMA	IR & asm pretraining	Compiler emulation	Flag tuning training	Disassembly	Round trips	Round trip BLEU
✓	✓	✓	✓	✓	49.4 % (-)	0.951 (-)
✓	✓	✓		✓	45.2 % (-8.5 %)	0.955 (+0.4 %)
✓	✓			✓	44.2 % (-10.5 %)	0.957 (+0.7 %)
✓				✓	39.0 % (-21.1 %)	0.965 (+1.5 %)
				✓	8.8 % (-82.8 %)	0.908 (-4.5 %)

Table 7. Performance at next-token prediction and compiler emulation tasks. For *Perplexity*, lower is better. For *Compiles* and *Exact match*, higher is better.

	Size	Perplexity		Compiler emulation	
		IR	Asm	Compiles	Exact match
CODE LLAMA	7B	1.456	1.423	5.4 %	1.2 %
	13B	1.429	1.404	4.8 %	0.8 %
LLM COMPILER	7B	1.052	1.046	87.0 %	16.0 %
	13B	1.047	1.043	95.6 %	20.0 %
LLM COMPILER FTD	7B	1.057	1.053	71.0 %	4.6 %
	13B	1.054	1.052	61.4 %	5.4 %

in this area are pretrained foundational models made available for others to build upon, including CODE LLAMA [46],

StarCoder [37], Magicode [58], DeepSeek-Coder [24], GPT-4 [40] and others [2, 14, 56]. Some of the existing models are open source [2, 37, 46, 58] while others are closed source [6, 23, 34, 40]. We extend the collection of foundational models for code with a family of models specifically trained on intermediate code representation with a license that allows wide reuse.

While LLMs have found broad adoption for coding tasks, few operate at the level of compilers. Gallagher et al. [18] train a RoBERTA architecture on LLVM-IR for the purpose of code weakness identification, and Transcoder-IR [51] uses LLVM-IR as a pivot point for source-to-source translation. Few LLMs include compiler IRs in their training, and of those that do, IRs comprise a tiny fraction of the data compared to other programming languages. StarCoder 2 [37] and DeepSeek-Coder [24] include 7.7 GB (0.4 %) and 0.91 GB

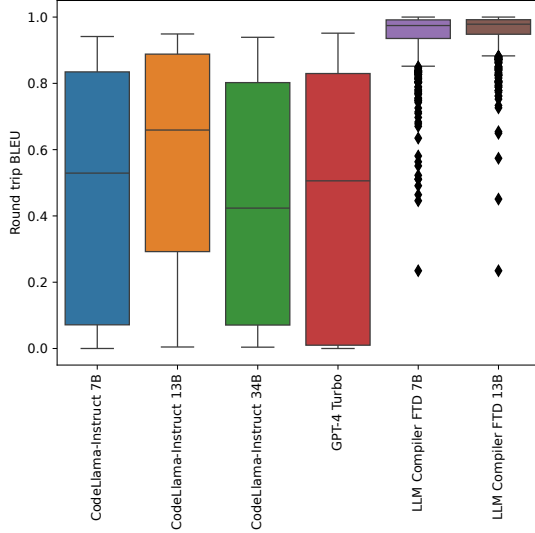


Figure 9. Distribution of round trip BLEU scores on the disassembly task.

(0.1 %) of LLVM-IR respectively in their training data. In contrast, LLM COMPILER is pretrained on 422 GB of LLVM-IR, and additional LLVM-IR during fine-tuning, and assembly code which makes up at least 85 % of the total training data. With the increasing interest in IR to improve the performance of code generation models, new datasets are emerging, for example, ComPILE [19], a 2.4 TB dataset of unoptimized LLVM-IR.

Language models have been used to perform program fuzzing [11, 59], test generation [48], automated program repair [60], and source-level algorithmic optimization Madaan et al. [38]. The introduction of fill-in-the-middle capabilities is especially useful for software engineering use cases such as code completion, and has become common in recent code models such as InCoder [15], SantaCoder [2], StarCoder [37], and CODE LLAMA [46]. A large number of useful applications have been explored for LLMs, however, only very few are directly focused on compilation tasks.

Paul et al. [42] create SLTrans, a 26 B token dataset which pairs high level source code with corresponding LLVM-IR. Like our dataset, they include different source languages and optimization levels for their IR, however, their optimization is limited to -Oz and -O3. They train IRCoder on 800 M tokens of SLTrans and demonstrate how it improves the code reasoning capabilities of underlying base models. IRCoder and StarCoder 2 present their models with LLVM-IR. We include both LLVM-IR as well as native assembly code from multiple source languages and for multiple architecture targets.

Many works have applied machine learning in compilers [4, 8, 33, 43, 49]. Compiler pass ordering has been exploited for decades. Over the years there have been several approaches using machine learning [1, 20, 29, 35, 39, 45].

Prior research works have compared LLM to non-LLM approaches [3, 9] for compiler optimization tasks and demonstrated how LLMs can outperform, for example, reinforcement learning or search based techniques even if given large search budgets. In [9] we compare an early version of LLM COMPILER against AutoPhase [26], a reinforcement learning approach that trains an agent to select the optimal sequence of optimization passes to maximize instruction count savings, and Coreset-NVP [35] which combines iterative search with a learned cost model to predict the best pass sequences and try the top ones. In this work we aim to compare against other LLMs and show how we improve upon them.

8 Limitations

We have shown that LLM COMPILER performs well at compiler optimization tasks and has improved understanding of compiler representations and assembly code over prior works, but there are limitations. The main limitation is the finite sequence length of inputs (context window). LLM COMPILER supports a 16k token context windows, but program codes may be far longer. For example, 67% of MiBench translation units exceeded this context window when formatted as flag tuning prompts. To mitigate this we split larger translation units into individual functions, though this limits the scope of optimization that can be performed, and still 18% of the split translation units remain too large for the model to accept as input. Researchers are adopting ever-increasing context windows [13], but finite context windows remain a common concern with LLMs.

A second limitation, common to all LLMs, is the accuracy of model outputs. Users of LLM COMPILER are advised to assess their models using evaluation benchmarks specific to compilers. Given that compilers are not bug-free, any suggested compiler optimizations must be rigorously tested. When a model decompiles assembly code, its accuracy should be confirmed through round trip, manual inspection, or unit testing. For some applications LLM generations can be constrained to regular expressions [21], or combined with automatic verification to ensure correctness [52].

9 Conclusions

We introduce a novel family of pre-trained large language models specifically designed to address the challenges of code and compiler optimization. We release LLM COMPILER under a bespoke commercial license to facilitate widespread access and collaboration, enabling both academic researchers and industry practitioners to explore, modify, and extend the model according to their specific needs. To further improve the performance and usability of LLM COMPILER, we aim to focus future research on addressing its limitations, particularly in handling long program codes, integrating more languages and compiler frameworks and targeting performance optimization objectives.

References

- [1] F. Agakov, E. Bonilla, J. Cavazos, B. Franke, G. Fursin, M.F.P. O’Boyle, J. Thomson, M. Toussaint, and C.K.I. Williams. 2006. Using Machine Learning to Focus Iterative Optimization. In *CGO*. <https://doi.org/10.1109/CGO.2006.37>
- [2] Loubna Ben Allal, Raymond Li, Denis Kocetkov, Chenghao Mou, Christopher Akiki, Carlos Munoz Ferrandis, Niklas Muennighoff, Mayank Mishra, Alex Gu, Manan Dey, Logesh Kumar Umapathi, Carolyn Jane Anderson, Yangtian Zi, Joel Lamy Poirier, Hailey Schoelkopf, Sergey Troshin, Dmitry Abulkhanov, Manuel Romero, Michael Lappert, Francesco De Toni, Bernardo García del Río, Qian Liu, Shamik Bose, Urvashi Bhattacharyya, Terry Yue Zhuo, Ian Yu, Paulo Villegas, Marco Zocca, Sourab Mangrulkar, David Lansky, Huu Nguyen, Danish Contractor, Luis Villa, Jia Li, Dzmitry Bahdanau, Yacine Jernite, Sean Hughes, Daniel Fried, Arjun Guha, Harm de Vries, and Leandro von Werra. 2023. SantaCoder: Don’t Reach for the Stars! *arXiv:2301.03988* (2023).
- [3] Jordi Armengol-Estapé, Jackson Woodruff, Chris Cummins, and Michael F.P. O’Boyle. 2024. SLDe: A Portable Small Language Model Decompiler for Optimized Assembler. In *CGO*. <https://doi.org/10.1109/CGO57630.2024.10444788>
- [4] Amir H Ashouri, Mostafa Elhoushi, Yuzhe Hua, Xiang Wang, Muhammad Asif Manzoor, Bryan Chan, and Yaoqing Gao. 2022. MLGPerf: An ML Guided Inliner to Optimize Performance. *arXiv:2207.08389* (2022).
- [5] Ying Cao, Ruigang Liang, Kai Chen, and Peiwei Hu. 2022. Boosting Neural Networks to Decompile Optimized Binaries. In *ACSAC*.
- [6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgén Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *arXiv:2107.03374* (2021).
- [7] Chris Cummins, Zacharias Fisches, Tal Ben-Nun, Torsten Hoefler, Michael O’Boyle, and Hugh Leather. 2021. ProGraML: A Graph-based Program Representation for Data Flow Analysis and Compiler Optimizations. In *ICML*.
- [8] Chris Cummins, Pavlos Petoumenos, Zheng Wang, and Hugh Leather. 2017. End-to-End Deep Learning of Optimization Heuristics. In *PACT*. <https://doi.org/10.1109/PACT.2017.24>
- [9] Chris Cummins, Volker Seeker, Dejan Grubisic, Mostafa Elhoushi, Youwei Liang, Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Kim Hazelwood, Gabriel Synnaeve, and Hugh Leather. 2023. Large language models for compiler optimization. *arXiv preprint arXiv:2309.07062* (2023).
- [10] Chris Cummins, Bram Wasti, Jiadong Guo, Brandon Cui, Jason Ansel, Sahir Gomez, Somya Jain, Jia Liu, Olivier Teytaud, Benoit Steiner, Yuandong Tian, and Hugh Leather. 2022. CompilerGym: Robust, Performant Compiler Optimization Environments for AI Research. In *CGO*. <https://doi.org/10.1109/CGO53902.2022.9741258>
- [11] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Zheyuan Yang, and Lingming Zhang. 2023. Large Language Models Are Zero-Shot Fuzzers: Fuzzing Deep-Learning Libraries via Large Language Models. In *ISSTA*. <https://doi.org/10.1145/3597926.3598067>
- [12] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient Finetuning of Quantized LLMs. *arXiv preprint arXiv:2305.14314* (2023).
- [13] Jiayu Ding, Shuming Ma, Li Dong, Xingxing Zhang, Shaohan Huang, Wenhui Wang, and Furu Wei. 2023. LongNet: Scaling Transformers to 1,000,000,000 Tokens. *arXiv:2307.02486* (2023).
- [14] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-trained Model for Programming and Natural Languages. *arXiv:2002.08155* (2020).
- [15] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. 2023. InCoder: A Generative Model for Code Infilling and Synthesis. *arXiv:2204.05999* (2023).
- [16] G. G. Fursin, M. F. P. O’Boyle, and P. M. W. Knijnenburg. 2005. Evaluating Iterative Compilation. In *LCPC*. https://doi.org/10.1007/11596110_24
- [17] Philip Gage. 1994. A New Algorithm for Data Compression. *C Users Journal* 12, 2 (1994).
- [18] Shannon K Gallagher, William E Klieber, and David Svoboda. 2022. LLVM Intermediate Representation for Code Weakness Identification.
- [19] Aiden Grossman, Ludger Paehler, Konstantinos Parasyris, Tal Ben-Nun, Jacob Hegna, William Moses, Jose M Monsalve Diaz, Mircea Trofin, and Johannes Doerfert. 2024. ComPile: A Large IR Dataset from Production Sources. *arXiv:2309.15432* (2024).
- [20] Dejan Grubisic, Chris Cummins, Volker Seeker, and Hugh Leather. 2024. Compiler generated feedback for Large Language Models. *arXiv:2403.14714* (2024).
- [21] Dejan Grubisic, Chris Cummins, Volker Seeker, and Hugh Leather. 2024. Priority Sampling of Large Language Models for Compilers. *arXiv:2402.18734* (2024).
- [22] Yi Gui, Yao Wan, Hongyu Zhang, Huifang Huang, Yulei Sui, Guandong Xu, Zhiyuan Shao, and Hai Jin. 2022. Cross-language binary-source code matching with intermediate representations. In *SANER*. <https://doi.org/10.1109/SANER53432.2022.00077>
- [23] Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, Adil Salim, Shital Shah, Harkirat Singh Behl, Xin Wang, Sébastien Bubeck, Ronen Eldan, Adam Tauman Kalai, Yin Tat Lee, and Yuanzhi Li. 2023. Textbooks Are All You Need. *arXiv:2306.11644* (2023).
- [24] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Quanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. *arXiv:2401.14196* (2024).
- [25] Matthew R Guthaus, Jeffrey S Ringenberg, Dan Ernst, Todd M Austin, Trevor Mudge, and Richard B Brown. 2001. MiBench: A free, commercially representative embedded benchmark suite. In *WWC*. IEEE. <https://doi.org/10.1109/WWC.2001.990739>
- [26] Ameer Haj-Ali, Qijing Huang, William Moses, John Xiang, John Wawrzyniec, Krste Asanovic, and Ion Stoica. 2020. AutoPhase: Juggling HLS Phase Orderings in Random Forests with Deep Reinforcement Learning. In *MLSys*.
- [27] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2023. Large Language Models for Software Engineering: A Systematic Literature Review. *arXiv:2308.10620* (2023).
- [28] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=nZeVKeeFYf9>
- [29] Tarindu Jayatilaka, Hideto Ueno, Giorgis Georgakoudis, EunJung Park, and Johannes Doerfert. 2021. Towards Compile-Time-Reducing Compiler Optimization Selection via Machine Learning. In *ICPP*. <https://doi.org/10.1109/ICPP53432.2021.00077>

- <https://doi.org/10.1145/3458744.3473355>
- [30] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A Survey on Large Language Models for Code Generation. *arXiv:2406.00515* (2024).
- [31] Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, Dzmitry Bahdanau, Leandro von Werra, and Harm de Vries. 2022. The Stack: 3TB of Permissively Licensed Source Code. *arXiv:2211.15533* (2022).
- [32] Chris Lattner and Vikram Adve. 2004. LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. In *CGO*. <https://doi.org/10.1109/CGO.2004.1281665>
- [33] Hugh Leather and Chris Cummins. 2020. Machine Learning in Compilers: Past, Present and Future. In *FDL*. <https://doi.org/10.1109/FDL50818.2020.9232934>
- [34] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-Level Code Generation with AlphaCode. *Science* 378, 6624 (2022). <https://doi.org/10.1126/science.abq1158>
- [35] Youwei Liang, Kevin Stone, Ali Shameli, Chris Cummins, Mostafa Elhoushi, Jiadong Guo, Benoit Steiner, Xiaomeng Yang, Pengtao Xie, Hugh Leather, and Yuandong Tian. 2023. Learning Compiler Pass Orders using Coreset and Normalized Value Prediction. In *ICML*.
- [36] Ilya Loshchilov and Frank Hutter. 2017. Decoupled Weight Decay Regularization. *arXiv:1711.05101* (2017).
- [37] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, Tianyang Liu, Max Tian, Denis Kocetkov, Arthur Zucker, Younes Belkada, Zijian Wang, Qian Liu, Dmitry Abulhanov, Indraneil Paul, Zhuang Li, Wen-Ding Li, Megan Risdal, Jia Li, Jian Zhu, Terry Yue Zhuo, Evgenii Zheltonozhskii, Nii Osae Osae Dade, Wenhao Yu, Lucas Krauß, Naman Jain, Yixuan Su, Xuanli He, Manan Dey, Edoardo Abati, Yekun Chai, Niklas Muennighoff, Xiangru Tang, Muh-tasham Oblokulov, Christopher Akiki, Marc Marone, Chenghao Mou, Mayank Mishra, Alex Gu, Binyuan Hui, Tri Dao, Armel Zebaze, Olivier Dehaene, Nicolas Patry, Canwen Xu, Julian McAuley, Han Hu, Torsten Scholak, Sebastien Paquet, Jennifer Robinson, Carolyn Jane Anderson, Nicolas Chapados, Mostofa Patwary, Nima Tajbakhsh, Yacine Jernite, Carlos Muñoz Ferrandis, Lingming Zhang, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. 2024. StarCoder 2 and The Stack v2: The Next Generation. *arXiv:2402.19173* (2024).
- [38] Aman Madaan, Alexander Shypula, Uri Alon, Milad Hashemi, Parthasarathy Ranganathan, Yiming Yang, Graham Neubig, and Amir Yazdanbakhsh. 2023. Learning Performance-Improving Code Edits. *arXiv:2302.07867* (2023).
- [39] William F. Oglvie, Pavlos Petoumenos, Zheng Wang, and Hugh Leather. 2017. Minimizing the Cost of Iterative Compilation with Active Learning. In *CGO*. <https://doi.org/10.1109/CGO.2017.7863744>
- [40] OpenAI. 2023. GPT-4 Technical Report. *arXiv:2303.08774* (2023).
- [41] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. BLEU: A Method for Automatic Evaluation of Machine Translation. In *ACL*. <https://doi.org/10.3115/1073083.1073135>
- [42] Indraneil Paul, Goran Glavaš, and Iryna Gurevych. 2024. IRCoder: Intermediate Representations Make Language Models Robust Multilingual Code Generators. *arXiv:2403.03894* (2024).
- [43] Phitchaya Mangpo Phothilimthana, Amit Sabne, Nikhil Sarda, Karthik Srinivasa Murthy, Yanqi Zhou, Christof Angermueller, Mike Burrows, Sudip Roy, Ketan Mandke, Rezsa Farahani, Yu Emma Wang, Berkin Ilbeyi, Blake Hechtman, Bjarke Roune, Shen Wang, Yuanzhong Xu, and Samuel J. Kaufman. 2021. A Flexible Approach to Auto-tuning Multi-pass Machine Learning Compilers. In *PACT*. <https://doi.org/10.1109/PACT52795.2021.00008>
- [44] LLVM PM. 2021. *Using the New Pass Manager — LLVM 17.0.6 documentation*. <https://llvm.org/docs/NewPassManager.html>
- [45] Nilton Luiz Queiroz Jr and Anderson Faustino da Silva. 2023. A graph-based model for build optimization sequences: A study of optimization sequence length impacts on code size and speedup. *COLA* 74 (2023).
- [46] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code Llama: Open Foundation Models for Code. *arXiv:2308.12950* (2023).
- [47] Eric Schulte, Jason Ruchti, Matt Noonan, David Ciarletta, and Alexey Loginov. 2018. Evolving Exact Decompilation. In *BAR*.
- [48] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2023. Adaptive Test Generation Using a Large Language Model. *arXiv:2302.06527* (2023).
- [49] Volker Seeker, Chris Cummins, Murray Cole, Björn Franke, Kim Hazelwood, and Hugh Leather. 2024. Revealing Compiler Heuristics Through Automated Discovery and Optimization. In *CGO*. <https://doi.org/10.1109/CGO57630.2024.10444847>
- [50] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing* 568 (2024). <https://doi.org/10.1016/j.neucom.2023.127063>
- [51] Marc Szafraniec, Baptiste Roziere, Francois Charton, Hugh Leather, Patrick Labatut, and Gabriel Synnaeve. 2022. Code Translation with Compiler Representations. *arXiv:2207.03578* (2022).
- [52] Jubi Taneja, Avery Laird, Cong Yan, Madan Musuvathi, and Shuvendu K. Lahiri. 2024. LLM-Vectorizer: LLM-based Verified Loop Vectorizer. *arXiv:2406.04693* (2024).
- [53] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Roziere, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. Llama: Open and efficient foundation language models. *arXiv:2302.13971* (2023).
- [54] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv:2307.09288* (2023).
- [55] Mircea Trofin, Yundi Qian, Eugene Brevdo, Zinan Lin, Krzysztof Choromanski, and David Li. 2021. MLGO: a Machine Learning Guided Compiler Optimizations Framework. *arXiv:2101.04808* (2021).
- [56] Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi DQ Bui, Junnan Li, and Steven CH Hoi. 2023. Codet5+: Open code large language models for code understanding and generation. *arXiv preprint arXiv:2305.07922* (2023).
- [57] Zheng Wang and Michael O’Boyle. 2018. Machine Learning in Compiler Optimisation. *arXiv:1805.03441* (2018).

- [58] Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. 2024. Magicoder: Empowering Code Generation with OSS-Instruct. *arXiv:2312.02120* (2024).
- [59] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2023. Universal Fuzzing via Large Language Models. *arXiv:2308.04748* (2023).
- [60] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated Program Repair in the Era of Large Pre-Trained Language Models. In *ICSE*. <https://doi.org/10.1109/ICSE48619.2023.00129>
- [61] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Zihan Wang, Lei Shen, Andi Wang, Yang Li, Teng Su, Zhilin Yang, and Jie Tang. 2023. CodeGeeX: A Pre-Trained Model for Code Generation with Multilingual Evaluations on HumanEval-X. *arXiv:2303.17568* (2023).

Received 2024-11-06; accepted 2024-12-21